

Introducción a Git & GitHub

Versionado de código

Comprender lo básico

¿Qué es **Git**?

Git es como un historial inteligente **para tu código**.

Guarda cada cambio que haces, te permite volver atrás si algo falla y trabajar con otros sin pisarse. Es como tener una máquina del tiempo y un organizador para tus proyectos.

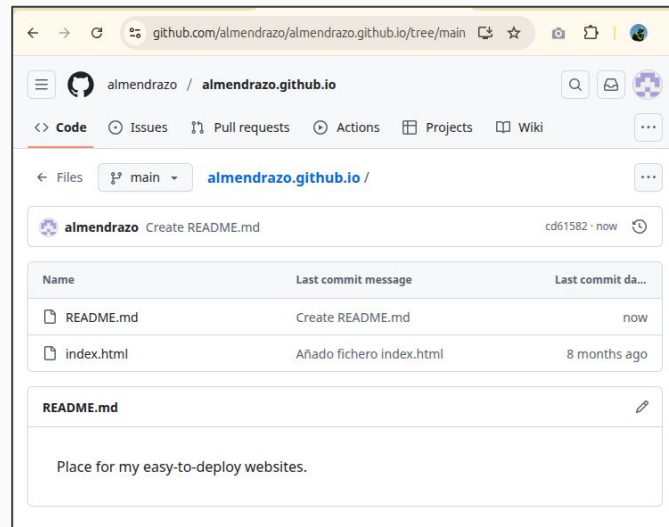


¿Qué es **GitHub**?

GitHub es **una plataforma** de Microsoft, donde subimos, compartimos y colaboramos en código.

Funciona sobre Git, así que gestiona el versionado de código.

GitHub es ideal para proyectos: desde pruebas personales hasta desarrollos profesionales en equipo.



¿Quién usa Git?

Git lo usa casi todo el mundo que **trabaja con código** hoy en día.

The Creator of Git and Linux



A random tech bro



Git se ha convertido en el estándar de versionado.
¿Vas a ser el único en la empresa que no sepa usarlo?

¿Cuánto cuesta Git?

Git es **completamente gratis**.

Es más, **GIT es FOS** (libre y de código abierto).

Y aunque no forma parte de GNU, su licencia es **GPLv2** (GNU General Public License versión 2); la cual exige que si distribuyes una versión modificada, también debe ser open source.

¿Para qué voy a usar Git?

✅ Guardar versiones del código

... y poder volver a un punto anterior si algo sale mal.

🤝 Colaborar en equipo

... y que varios desarrollen al mismo tiempo sin pisarse los cambios.

🔍 Ver el historial de cambios

... y saber quién cambió qué línea, cuándo y hasta por qué.

🧪 Probar sin romper

... y crear ramas para trabajar en nuevas funciones sin afectar el proyecto principal.

☁ Sincronizar con la nube

... y subir el repositorio a plataformas:
*GitHub, **GitLab**, Bitbucket,...*

Kahoot!

Son enlaces al mismo Kahoot!

<https://is.gd/0gc3Ak>

<https://create.kahoot.it/details/c7a7e666-f23c-466f-9e22-a33df9d65ca7>

Instalando

¿De dónde me bajo Git?

apt-get install git

De la página de Git: <https://git-scm.com>

en su sección de descargas: <https://git-scm.com/downloads>

Tienes versiones para Windows, Mac y Linux.

Primer contacto

¿Había dicho ya que Git es una herramienta de la línea de comandos?
Pues vamos a abrir una shell y lanzar el primer comando.

```
git --version # primer comando, veo si se instaló bien
```

Configuro mi usuario de Git

Usuario y correo, necesarios para registrar quién hace cambios.

```
git config --global user.name "Juan Anónimo"
```

```
git config --global user.email "janonimo@colegio.es"
```

```
git config --list
```

(OPCIONAL) Editor de texto asociado a Git.

```
git config --global core.editor "code --wait"
```

(OPCIONAL) Activar colores en la terminal.

```
git config --global color.ui auto
```

Herramientas gráficas (GUI)

La línea de comandos es la forma más poderosa y flexible de usar GIT
... pero podría instalarme interfaces gráficas como:

- **Sourcetree** (de Atlassian) - <https://www.sourcetreeapp.com/>
- **GitKraken** - <https://www.gitkraken.com/git-client>
- **Fork** - <https://git-fork.com/>

O la extensión **GitLens** para VSCode.

Primer repositorio

Vamos a crear las carpetas de mi proyecto

Cread un script dentro de una carpeta nueva: `proyecto1/hola.bat`

```
mkdir proyecto1  
cd proyecto1  
notepad hola.bat
```

```
REM Este script bat muestra un saludo  
echo Hola mundo
```

Iniciar un repositorio Git

Dentro de la carpeta `proyecto1`

```
git init
```

```
dir /a      # comando de la shell para ver fichs. ocultos
```

```
git status
```


Ver el estado de un repositorio Git

```
git status
```

```
C:\Users\Usuario_008\Documents\repositorio-git\proyecto1>git status
```

```
On branch main
```

```
No commits yet
```

```
Untracked files:
```

```
(use "git add <file>..." to include in what will be committed)
```

```
hola.bat
```

```
nothing added to commit but untracked files present (use "git add" to track)
```

¿En qué situación estoy?

Entendámoslo así: vamos a crear un álbum de fotos de mi código.

Acabo de comprar el álbum para toda la carpeta `proyecto1`

Todavía no tengo ninguna foto en el álbum.

Pero tengo un fichero `hola.bat` que podría formar parte de la foto

Git por dentro

La carpeta oculta `.git`

Al ejecutar `git init` dentro de la carpeta `proyecto1`:

- Git crea la carpeta oculta `.git`, que es el corazón del proyecto
 - el historial de commits
 - las ramas
 - el área de preparación (*staging area*)
 - las configuraciones locales del repositorio
- Git apunta a la rama por defecto (normalmente *main* o *master*)

Contenido de la carpeta oculta `.git`

- ♦ `objects/`

Elementos que Git rastrea (blobs, trees y commits) comprimidos y codificados (hash SHA-1).

- ♦ `refs/`

Contiene referencias a ramas, etiquetas (tags) y HEAD (la rama actual).

- ♦ `HEAD`

Un archivo que apunta a la rama actual (ej: refs/heads/main).

- ♦ `index`

Es el **staging area**: el área de preparación donde decides qué incluir en el siguiente commit.

Ejemplo

Si hago tres commits (tres fotos):

`Commit A` → `snapshot` con archivos: `index.html`, `style.css`

`Commit B` → solo cambias `index.html`

`Commit C` → cambias `style.css`

- 2 blobs para `index.html`
- 2 blobs para `style.css`
- 3 commits, cada uno apuntado a una tree con referencias a esos blobs

Mi primer commit

¿Qué es un commit?

Es hacerle una foto a mi proyecto.

El commit se guarda en el repositorio = la foto va al álbum.

¿Qué pasa por dentro al hacer un commit?

1. He modificado archivos en el proyecto.
2. Haces `git add` → Git guarda blobs en el índice (index) y los prepara para el commit.
3. Haces `git commit` → Git:
 - a. Crea un objeto tree con la estructura de carpetas.
 - b. Crea un commit que apunta a ese tree.
 - c. Asigna un hash SHA-1 único al commit.
 - d. Actualiza el puntero HEAD a ese nuevo commit.

Procedo a hacer **commit**

Dentro de la carpeta **proyecto1**

```
git add . # añado todos los ficheros a la stage area
git status
git log --oneline
```

```
git commit -m "Primer commit" -m "Crea script hola.bat"
git status
git log --oneline
```

Mi segundo commit

Hago cambios en mi proyecto

Dentro de la carpeta `proyecto1`

```
notepad hola.bat # cambio el mensaje de hola
```

```
notepad adios.bat # creo un script nuevo
```

```
git status
```

Veo los cambios en el repositorio

```
C:\Users\Usuario_008\Documents\repositorio-git\proyecto1>git status
On branch main

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
        new file:   hola.bat

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   hola.bat

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        adios.bat
```

Subo los cambios al stage

Dentro de la carpeta `proyecto1`

```
git add .  
git status
```

Hago el commit

```
git commit -m "Añado script adios.bat"
```

```
git status
```

```
git log --oneline --graph --all
```

Saber algo del commit

Ver contenido de un commit

Cada commit tiene asociado un **sha-1**, un código único (p.ej **a1b2c3d**) y puedo ver el contenido de un commit con `git show`

```
git show a1b2c3d
```

Comparar un commit con otro

Puedo comparar el contenido de dos commits.

O incluso el de un commit con mi working directory, el estado en el que tengo el proyecto actualmente.

```
git diff a1b2c3d z44t7g
```

```
git diff a1b2c3d
```

Traerte un fichero de otro commit

Es fácil deducir lo útil que es poder traerse un fichero de otro commit.
Este truco lo he usado en múltiples ocasiones para arreglar problemas.

```
git checkout a1b2c3d -- ruta/del/archivo
```

también con:

```
git restore --source=a1b2c3d ruta/del/archivo
```

Branching

Qué son las ramas

Una rama o **branch** es un puntero a un commit.

Cuando creo una rama nueva, estoy creando un puntero nuevo.

Sirven para:

- Desarrollar nuevas funcionalidades sin romper el código principal.
- Probar ideas o hacer correcciones temporales.
- Colaborar: cada persona puede trabajar en su propia rama.
- Manejar versiones: producción, pruebas, desarrollo, etc.

Comandos de creación de ramas

```
git branch rama-cambios # crear una rama
```

```
git checkout rama-cambios # cambio de mi rama actual a rama-cambios
```

```
git checkout -b rama-cambios # crear y cambiar en un solo paso
```

Comandos de gestión de ramas (1)

```
git branch --show-current # dime en qué rama estoy ahora
```

```
git branch      # lista todas las ramas locales
```

```
git branch -a   # lista todas las ramas: locales y remotas
```

```
git branch -d rama-alternativa # elimina una rama
```

```
git branch -D rama-alternativa # elimina una rama a la fuerza
```

```
git branch -m nuevo-nombre      # cambia el nombre de mi rama actual
```

Comandos de gestión de ramas (2)

Esto es de lo más importante: fusionar ramas.

```
git merge otra-rama # fusiona una rama con la actualmente activa
```

```
git diff rama1..rama2 # diferencias entre ramas
```

```
git push origin rama1 # sube al remoto la rama "rama1"
```

```
git push -u origin rama1 # se usa la 1ª vez que se sube "rama1" al remoto
```


Aprender branching jugando

Y ahora vamos a aprender branching de la misma manera que lo hice yo: divirtiéndonos con la mejor página que existe para esto:

<https://learngitbranching.js.org>

GitHub

Qué es GitHub

GitHub es una **plataforma web** para **alojar proyectos de código** que usan el sistema de control de versiones Git.

<https://github.com>

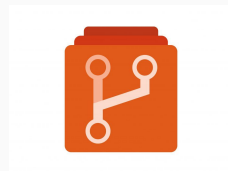
Desarrolladores y equipos colaboran, comparten y gestionan proyectos de software, tanto públicos como privados.

Qué utilidad cumple GitHub

- Subir y compartir código
- Controlar versiones con Git
- Revisar y discutir cambios en el código (pull requests)
- Gestionar tareas (issues, projects)
- Alojamiento de documentación (README, Wiki)
- Publicar páginas web (GitHub Pages)

Alternativas a GitHub

GitHub no es el único repositorio Git usado a nivel empresarial. Alternativamente encontramos: GitLab, Bitbucket, SourceForge, Azure DevOps (Repos),...



Crear cuenta en GitHub

Crear cuenta en GitHub es muy sencillo:

1. vamos a la página de GitHub: <https://github.com>
2. le damos al botón "Sign Up"
3. introduce nombre de usuario **smr2hip25-<nombre><1ª letra apellido>**
4. introduce el correo del cole

Hacer fork a un proyecto

Forkear un proyecto no es más que duplicar un proyecto de GitHub en mi cuenta.

A partir de ahí, puedo modificar mi nuevo proyecto como quiera.

Como ejercicio, haz fork a este proyecto en tu cuenta:

<https://github.com/firstcontributions/first-contributions>

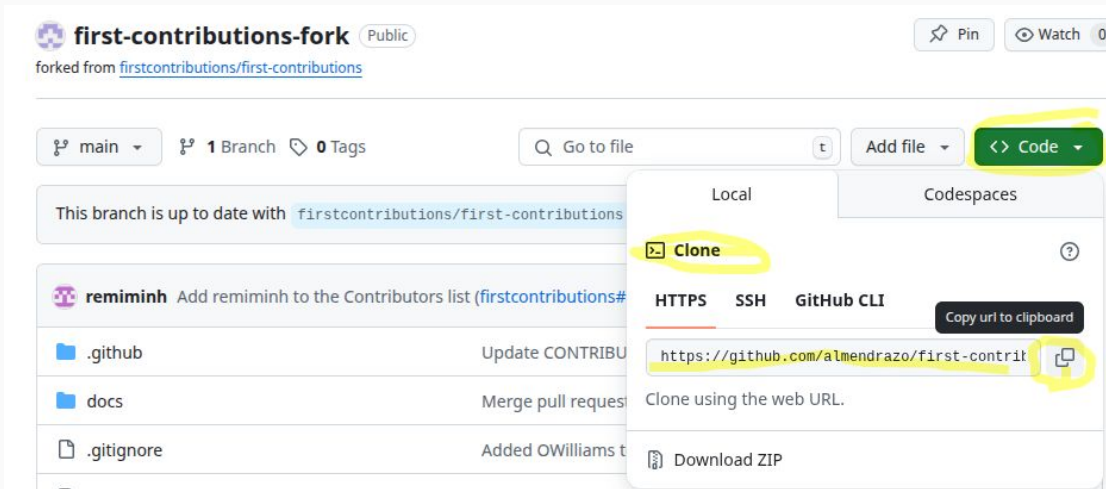


Clonar un proyecto en local

De GitHub puedo clonar mis proyectos y también los de otros.

Los modificaré en local
cuanto me apetezca.

Pero para subir los cambios
a GitHub, sólo podré si
tengo permisos de push.



Subir un repositorio local

1. Crea un repositorio en GitHub (repo-gh)
2. Desde la línea de comandos de tu ordenador:
 - a. `git remote add origin https://github.com/smr2h25_xxx/repo-gh.git`
 - b. `git checkout -M main` # si mi rama actual es master
 - c. `git push -u origin main`

Y un último consejo

Los comandos habituales de Git apréndetelos porque será los que uses el 85% del tiempo.

¿El otro 15%? Usa Google.
Si los necesitas de verdad, los irás absorbiendo con el uso, pero no te obsesiones con sabértelo todo porque Git es muy profundo.



¡Gracias!

1º FP GM SMR CEIPS Hipatia

raveigab@colegiohipatia.fuhem.es

